# Q-learning and DQN: Representation, Stability, and Sensitivity

John Quinchuqui
Intelligent Agents

CartPole-v1 and Acrobot-v1 | Individual Project 01

## Abstract

This study compares tabular Q-learning and DQN in CartPole-v1 and examines DQN in Acrobot-v1 through two hyperparameter sweeps and an ablation. Each main configuration uses three seeds, with equal budgets of 200,000 steps in CartPole and 300,000 in Acrobot. Periodic evaluation uses ten episodes without exploration. Final evaluation uses one hundred episodes per seed.

In CartPole, Q-learning with 20 bins achieved $252.29 \pm 142.32$, and the original DQN achieved $87.94 \pm 0.93$. Subsequent DQN tuning raised the mean to $332.50 \pm 137.42$, but only one seed reached 475. In Acrobot, the base configuration achieved $-83.03 \pm 5.55$ and met the threshold in 3/3 seeds. Without a target network, all runs ended at $-500$. Bands and $\pm$ values show the standard deviation across seeds rather than confidence intervals.

The results show that successful execution does not guarantee stable learning. Periodic peaks, final performance, and variation across seeds require separate analysis. The study retains negative results and states the limits of its conclusions.

**Keywords.** Q-learning, DQN, discretization, exploration, target network, reproducibility.

## 1. Introduction

The goal is to evaluate how state representation and training decisions affect performance, time efficiency, and stability. The study asks three questions. Which agent learns better under the same interaction budget? How sensitive are the agents to tabular granularity and DQN hyperparameters? What changes when the target network is removed?

This individual project covers Tier 1, Tier 2, and the written analysis of excluded environments. Tier 3 and ready-made agents from external libraries are outside the scope.

## 2. Suitability of the environments

CartPole has four continuous observations and two discrete actions. Acrobot has six observations and three actions [1, 2]. Both allow a network with one output per action and can be simulated on a CPU. Their numerical returns use different scales and each environment has its own success criterion.

| Environment | Task and project criterion |
|---|---|
| CartPole-v1 | Maintain balance. Final mean $\geq 475$ over 100 episodes. |
| Acrobot-v1 | Raise the tip of the articulated system. Final mean $\geq -100$ over 100 episodes. |

CartPole suits tabular study because each observation can be assigned to a cell in a four-dimensional grid. Acrobot requires coordination over time to build up energy. Its reward penalizes the time taken to reach the goal, so a less negative return indicates a quicker solution [2].

## 3. Methods: Tabular Q-learning

Each CartPole variable is divided uniformly into 6, 10, or 20 intervals. All three variants use the same ranges: position $\pm 2.4$, linear velocity $\pm 3.0$, angle $\pm 0.2095$, and angular velocity $\pm 3.5$. These ranges are choices made for the discretizer. The velocity ranges are not mandatory physical limits.

Values outside the range are assigned to the outer bins. The table is initialized to zero and holds two values per cell. The greedy action uses the first maximum, and the training policy adds random exploration. The implemented update is

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma(1-d)\max_{a'} Q(s',a') - Q(s,a)\big].$$

The settings are $\alpha = 0.1$ and $\gamma = 0.99$. The variable $d$ indicates true termination. This update estimates discounted return, while the report also evaluates the undiscounted episodic return required by the project.

### 3.1. Representation and capacity

| Bins per variable | Cells | Q-values | Memory |
|---|---|---|---|
| 6 | 1,296 | 2,592 | 10.1 KiB |
| 10 | 10,000 | 20,000 | 78.1 KiB |
| 20 | 160,000 | 320,000 | 1.22 MiB |

The sizes use $b^4$ cells, two actions, and four bytes per float32 value, excluding auxiliary objects. Increasing the number of bins from 6 to 20 multiplies the number of cells by about 123.5. Memory remains small, but the interaction budget must cover more regions. Some cells may be unreachable or unvisited.

### 3.2. DQN implementation

The network is a multilayer perceptron with two hidden layers of 64 ReLU units. It takes the observation vector and produces two values in CartPole or three in Acrobot. The replay buffer stores up to 50,000 transitions. After 1,000 warm-up steps, the agent samples 64 transitions without replacement and updates every four environment steps.

The implementation uses Adam, Huber loss, and gradient norm clipping at 10. The target network starts as a copy of the online network and is synchronized every 1,000 steps in the base configuration. The target is computed without tracking gradients.

$$y = r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a'),$$
$$L = \text{Huber}\left(Q_{\text{online}}(s, a), y\right).$$

The loss trains the prediction for the action taken. Replay reuses experience and reduces immediate dependence between consecutive samples. The target network temporarily separates the prediction being updated from the value used as a reference. Both mechanisms can support stability, while convergence remains an empirical question.

The ablation keeps replay, architecture, optimizer, and exploration policy fixed. It uses $Q_{\text{online}}$ in place of $Q_{\text{target}}$ to compute $y$, still without gradients. The experiment isolates the target network and keeps replay active. Double DQN is outside this ablation.

## 4. Experimental protocol

| Component | Setting |
|---|---|
| Seeds | 11, 22, and 33 |
| CartPole / Acrobot | 200,000 / 300,000 steps |
| Periodic evaluation | 10 episodes every 10,000 steps, including step 0 |
| Final evaluation | 100 episodes with different seeds |
| Base Adam rate | 0.001 |
| Discount / batch size | 0.99 / 64 |
| Replay / warm-up | 50,000 / 1,000 |
| Base exploration | $\epsilon$ falls from 1.0 to 0.05 over half the budget |

Exploration decays linearly and then stays at 0.05. The positive exploration floor prevents the policy from satisfying GLIE. Training and evaluation use separate environments. During evaluation, the agent always takes the greedy action, with $\epsilon = 0$.

Each configuration fixes the Python, NumPy, PyTorch, and environment seeds. Periodic evaluations for a given seed reuse initial conditions to make comparisons over time easier. Final evaluation uses a different set, calculated as $900000 + 1000 \times \text{seed} + \text{episode}$.

True termination disables bootstrapping. Truncation at the time limit resets the episode while allowing the agent to estimate future value, consistent with the API distinction [3]. The code stores training returns separately in `training.csv`. The main curves use `evaluation.csv`.

The figures show the mean and one sample standard deviation across three seeds. The first threshold crossing in ten episodes is called a **provisional crossing**. It establishes neither final success nor sustained performance. Final success requires the mean of one hundred episodes. Evaluation uses the model at the last training step.

The main experiment matrix contains 30 training runs and 7.8 million steps. Four additional diagnostic runs add 0.8 million steps. Smoke tests are excluded from the experimental results.

## 5. Tier 1 results

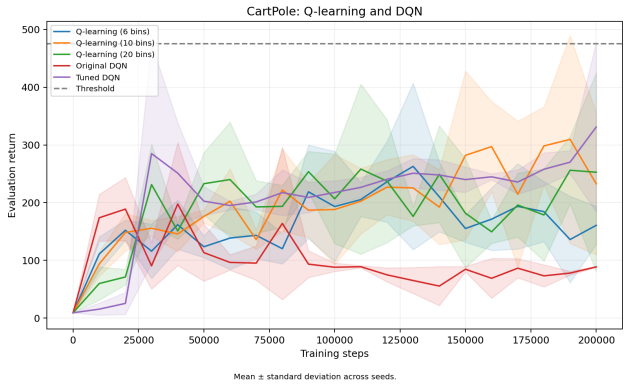



**Figure 1:** Periodic evaluation in CartPole with the same budget per run. Tuned DQN appears as a follow-up analysis alongside the original DQN.

| Method | Final mean ± SD | Successes | s/run |
|---|---|---|---|
| Q 6 | $177.17 \pm 19.74$ | 0/3 | 9.1 |
| Q 10 | $233.69 \pm 139.99$ | 0/3 | 9.9 |
| Q 20 | $252.29 \pm 142.32$ | 0/3 | 9.6 |
| Base DQN | $87.94 \pm 0.93$ | 0/3 | 202.8 |
| Tuned DQN | $332.50 \pm 137.42$ | 1/3 | 207.7 |

No tabular variant reached 475 in final evaluation. The highest tabular mean came from 20 bins, although its SD of 142.32 shows considerable variation across seeds. With 10 bins, final returns were 391.51, 185.02, and 124.53. With 20 bins, they were 187.80, 415.44, and 153.64. Improvement was not monotonic across bin counts within each seed.

Finer discretization reduces local aggregation but demands greater coverage. Saturation in the outer bins and a constant $\alpha$ may contribute to policy fluctuations. These explanations are compatible with the curves. Cell occupancy and visit counts were not measured, so their effects cannot be isolated.

**Training returns with exploration**

| Training configuration | Mean | Seed SD |
|---|---|---|
| Q 6 | 174.95 | 26.04 |
| Q 10 | 241.89 | 91.31 |
| Q 20 | 207.47 | 88.66 |
| CP base DQN | 57.44 | 6.97 |
| CP tuned DQN | 282.66 | 34.17 |
| Ac base | −94.29 | 5.71 |
| Ac low rate | −92.51 | 3.04 |
| Ac high rate | −120.90 | 36.58 |
| Ac fast $\epsilon$ | −90.96 | 3.95 |
| Ac slow $\epsilon$ | −95.12 | 3.20 |
| Ac no target | −385.62 | 175.23 |

For each seed, the return is averaged over episodes completed in the last 10,000 steps ($\epsilon = 0.05$), then averaged across seeds. Episode counts can differ by seed. This table describes exploratory training. CP denotes CartPole and Ac denotes Acrobot.

### 5.1. Follow-up DQN diagnostics

The original DQN finished at 87.02, 87.92, and 88.88 despite temporary improvements. Its low final SD reflects consistently low performance. Reducing only the learning rate to 0.0001 for seed 11 yielded 121.49 over one hundred episodes. The isolated run did not resolve the deterioration.

With the lower learning rate held fixed, the target network copy interval was increased to 10,000 steps, and all three seeds were run. Final returns were 490.25, 238.81, and 268.44. The mean rose to 332.50, but only seed 11 met the criterion. The improvement of 244.56 points over the original configuration applies to these runs.

| Seed | First provisional crossing | Final |
|---|---|---|
| 11 | 200,000 steps | 490.25 |
| 22 | 30,000 steps | 238.81 |
| 33 | Not reached | 268.44 |

Seed 22 reached 500 over ten episodes at 30,000 steps and finished below the target. This shows why a first crossing is insufficient for stopping training and declaring the task solved. The saved final weights cannot automatically recover the earlier peak.

### 5.2. Efficiency and fair comparison

The tables averaged 9.12 to 9.85 s per run, compared with 202.76 s for base DQN and 207.73 s for tuned DQN. At the same number of interactions, Q-learning required substantially less computation time. Tuned DQN had the highest final mean of the evaluated methods, at greater time cost and with high dispersion.

There is no robust winner in time to solve CartPole because no method met the threshold across all seeds. Tuning followed observation of the initial failures, so DQN received a larger search budget than the tabular variants. All three tuned results are reported. The single-seed trial serves as a diagnostic.

## 6. Tier 2: Learning rate

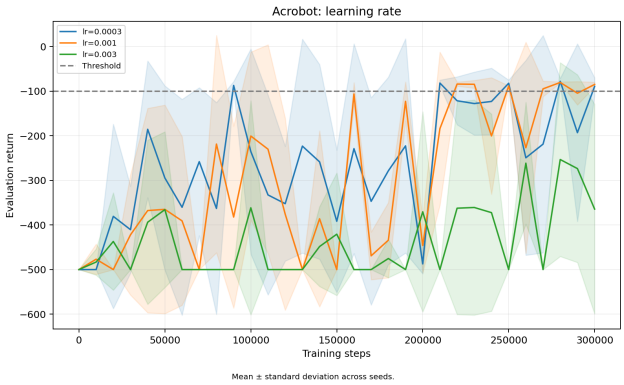



**Figure 2:** Acrobot with only the Adam learning rate varied. Bands represent dispersion across seeds.

| Rate | Final mean ± SD | Successes |
|---|---|---|
| 0.0003 | $-84.23 \pm 1.21$ | 3/3 |
| 0.001 | $-83.03 \pm 5.55$ | 3/3 |
| 0.003 | $-363.42 \pm 236.56$ | 1/3 |

The base rate produced the highest final mean in the sweep, only 1.20 points above the low rate. With three seeds and no inferential test, the comparison is descriptive. The low rate showed less final dispersion. Temporal stability remains a separate question.

At 0.003, seeds 11 and 33 ended at $-500$, after reaching their best periodic returns of $-87.0$ and $-82.2$. The agents therefore displayed effective policies before deteriorating. Seed 22 ended at $-90.26$. Return logs alone cannot identify the exact cause.

An aggressive update may shift predictions too far relative to targets that are themselves changing, while replay mixes experience from earlier policies. This hypothesis fits the deterioration. Distinguishing numerical instability, overestimation, and other mechanisms would require losses, Q-value magnitudes, and gradients.

**First provisional crossing in Acrobot**

| Variant | Steps for seeds 11 / 22 / 33 |
|---|---|
| Base | 80k / 50k / 80k |
| Low rate | 50k / 90k / 40k |
| High rate | 220k / 300k / 100k |
| Fast $\epsilon$ | 40k / 80k / 110k |
| Slow $\epsilon$ | 160k / 200k / 140k |
| No target | 210k / not reached / not reached |

Each k denotes 1,000 steps. Crossings use ten episodes and are provisional. The tabular variants and original DQN had no crossings in CartPole.

### 6.1. Exploration duration

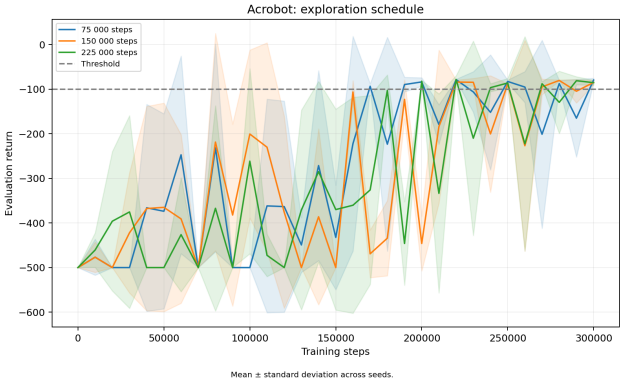



**Figure 3:** Acrobot exploration schedules. The value of $\epsilon$ reaches 0.05 after 75,000, 150,000, or 225,000 steps. The total budget is always 300,000 steps.

| Fraction | Final mean ± SD | Successes |
|---|---|---|
| 0.25 | $-87.11 \pm 13.57$ | 2/3 |
| 0.50 | $-83.03 \pm 5.55$ | 3/3 |
| 0.75 | $-102.89 \pm 22.48$ | 2/3 |

Fast exploration decay yielded $-78.47$, $-80.10$, and $-102.75$. Slow decay yielded $-127.68$, $-83.83$, and $-97.17$. The middle duration was the only setting that succeeded in all three seeds and had the highest final mean among these variants.

Exploring for less time may reduce early diversity and exploit an incomplete policy. Exploring longer also reduces the number of greedy interactions within the same budget. The effect depends on the environment, the replay buffer, and the distribution of visited states.

The rate and exploration schedule were varied separately. The two sweeps share three base runs and change one variable at a time. Five configurations cover the two sweeps of three values. The design cannot estimate interactions or identify the best joint combination.

The studies keep tabular ranges, network width, buffer capacity, and discount fixed. The conclusions apply to the documented architecture, budgets, and three seeds.

## 7. Target-network ablation

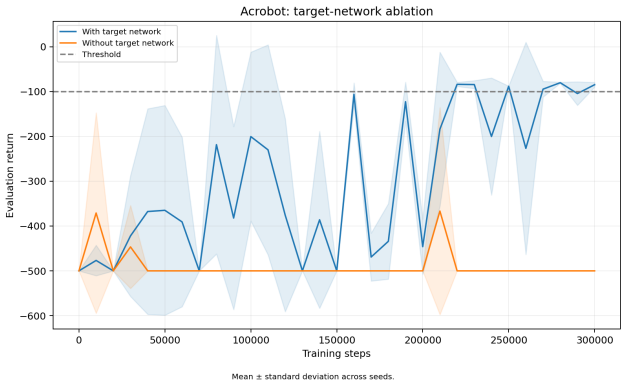



**Figure 4:** Base Acrobot DQN compared with the variant without a target network. Replay and the remaining parameters stay fixed.

| Variant | Final mean ± SD | Successes |
|---|---|---|
| With target | $-83.03 \pm 5.55$ | 3/3 |
| Without target | $-500.00 \pm 0.00$ | 0/3 |

Without a target network, all three seeds reached the time limit in all one hundred final episodes. The zero SD reflects the same failure outcome across runs. Seed 11 provisionally crossed $-100$ at 210,000 steps with $-99.9$, then lost that performance.

The result supports temporarily decoupling the target in this implementation. When the same network predicts values and constructs the reference, each update changes both quantities and can amplify errors. The ablation measures the effect under fixed conditions. Other implementations without target networks may behave differently.

Replay was retained. Its causal contribution cannot be quantified from this ablation. Keeping replay constant allows the experimental contrast to be attributed to the change in the target, subject to random variation and the limits of the design.

### 7.1. Time, hardware, and evidence

| Acrobot configuration | Total s/run | s/run excl. eval. |
|---|---|---|
| Base | 234.9 | 222.9 |
| Low rate | 163.6 | 156.3 |
| High rate | 174.5 | 160.4 |
| Fast exploration decay | 167.1 | 159.3 |
| Slow exploration decay | 165.8 | 157.0 |
| Without target | 171.1 | 154.7 |

The 30 main runs total 3,925.08 s (65 min 25 s). Diagnostics add 900.29 s (15 min). The recorded total is 80 min 25 s. These are sums of `wall_seconds` per run and exclude installation, smoke tests, breaks, debugging, plotting, and writing.

The reported computer is a Lenovo LOQ with an Intel Core i5-13450HX, 24 GB of RAM, and an RTX 5050 with 8 GB. The logs confirm Windows build 26300, Python 3.11.9, and CPU execution with one PyTorch thread. The installed versions are Gymnasium 1.2.3, PyTorch 2.15.0+cpu, and NumPy 2.4.6. The runs used the CPU. The value `gpu=null` in the log describes the execution configuration.

Time excluding evaluation still includes interaction with the environment, writing, and saving. Differences between runs may reflect evaluation, episode length, load, or temperature. Power consumption and hardware utilization were not measured.

#### Qualitative evidence

Using a fixed initial condition different from those used in evaluations, the GIF of tuned CartPole DQN with seed 11 returned 331. The GIF of base Acrobot DQN with seed 11 returned $-75$. Both files open and show one illustrative episode. The one-hundred-episode evaluations remain the basis of the aggregate results.

CartPole illustrates a limitation even for the successful seed: some initial conditions terminate before 500. Acrobot shows a trajectory consistent with the observed final mean. The GIF filenames identify the environment, variant, and seed.

## 8. Analysis of excluded environments

### 8.1. Continuous actions and DQN

BipedalWalker and MuJoCo locomotion tasks were excluded because of the relationship between representation, the number of actions, and the learning budget. They can be simulated on a laptop. Conventional DQN produces one value per discrete action and cannot directly maximize over a continuous vector of torques.

The documented action dimensions are four for BipedalWalker, three for Hopper, six for HalfCheetah, and eight for Ant [4]. If each dimension is replaced by $k$ levels and every combination is enumerated, the number of joint actions is $k^d$.

| Environment | $d$ | $k = 3$ | $k = 5$ | $k = 11$ |
|---|---|---|---|---|
| Hopper | 3 | 27 | 125 | 1,331 |
| BipedalWalker | 4 | 81 | 625 | 14,641 |
| HalfCheetah | 6 | 729 | 15,625 | 1,771,561 |
| Ant | 8 | 6,561 | 390,625 | 214,358,881 |

For Ant with eleven levels, a network whose last hidden layer has 64 units would require $65 \times 214{,}358{,}881 = 13{,}933{,}327{,}265$ output-layer parameters, including biases. These parameters alone would occupy about 55.7 decimal GB in float32. Gradients, Adam state, and a target network would multiply the cost and are excluded from that estimate.

Even with five levels, Ant would need 390,625 outputs and about 25.4 million parameters in that layer. A network that fits in memory still needs data to distinguish a vast number of combinations. Each experience updates the prediction for the selected action.

Coarse discretization saves some of this cost at the expense of control precision. Independent quantization can discard useful torque settings or smooth coordination. The $k^d$ count applies to the full Cartesian grid. Factored parameterizations can change the cost and would require a different design from the enumerative DQN implemented here.

### 8.2. Why a table is also unsuitable

BipedalWalker has 24 observation components [4]. With only six bins per component, a full grid contains $6^{24} \approx 4.74 \times 10^{18}$ cells. If its four action dimensions are each discretized into three levels, there would be 81 joint actions and approximately $3.84 \times 10^{20}$ Q-values. These would require $1.54 \times 10^{21}$ bytes in float32.

The calculation describes the cost of a direct Cartesian table and does not assume that every state is reachable. A sparse table avoids allocating unvisited states, but it still needs to explore relevant combinations and lacks generalization on its own. Physically similar states can fall in different cells and receive separate updates.

### 8.3. Suitable algorithm families

**PPO** learns a parameterized policy and supports continuous actions. Its clipped objective limits the incentive to change the policy too far in one update. It is a reasonable alternative for locomotion when enough interactions are available. PPO is on-policy and does not reuse a historical replay buffer indefinitely as DQN does [5].

**SAC** uses a stochastic policy and off-policy learning. Entropy regularization encourages action diversity while critics estimate value. The policy produces continuous vectors without enumerating $k^d$ combinations. Transition reuse can help when simulation is expensive, and performance still depends on tuning [6].

**TD3** uses a deterministic continuous actor, two critics, delayed policy updates, and target action smoothing. It uses the smaller critic estimate when building the target to reduce exploitation of excessive estimates. It provides another option for continuous control with replay [7].

The choice depends on interaction cost, stability, exploration, and resources. The project discusses these families as required. No agents from these families were trained, and no experimental results are assigned to them. Discretizing one torque dimension, as in Pendulum, is more manageable than discretizing eight joints together.

### 8.4. Visual observations in Atari

A visual observation can make tabular learning infeasible even with few actions. A single $84 \times 84$ grayscale image contains 7,056 pixels. With 256 possible values per pixel, there are $256^{7056}$ possible images, approximately $10^{16992}$ combinations. This is a combinatorial bound on representation. The number of physically reachable scenes is smaller.

Stacking four images produces 28,224 values and $256^{28224}$ possible combinations, on the order of $10^{67968}$. Even binary pixels would leave $2^{28224}$ potential inputs. A table indexed by image cannot visit enough of them or transfer learning between similar scenes.

Moving an object a few pixels can change a table key even when the decision is nearly identical. A static image can also hide the speed or direction of motion. Excessively coarse manual aggregation can discard the very information needed to act.

Neural DQN can handle images with discrete actions. Convolutional filters are shared across positions and extract spatial patterns. Stacking frames adds temporal information. Early DQN work on Atari used a convolutional network and replay to learn from pixels [8]. The dense network in this project was designed for four or six input values, and Atari calls for a different input design.

A transition storing separate current and next $84 \times 84 \times 4$ states in uint8 uses 56,448 bytes, excluding actions and metadata. One million such transitions would use 56.45 GB, or 225.79 GB in float32. Reusing frames and reconstructing stacks reduces this cost. Buffer design therefore matters alongside GPU capacity.

Atari remains outside the experimental scope. The calculations in this section characterize representations, and the project provides no measured comparison of CNN, PPO, SAC, or TD3 agents.

## 9. Discussion and limits

**Which method learned faster in Tier 1?** The answer depends on the metric. Q-learning improved early and took about ten seconds per run, while DQN took several minutes. No method met the strict criterion of solving CartPole across all seeds. Tuned DQN had a higher final mean and received an additional search budget.

**What sensitivity was observed?** More bins raised the highest tabular mean without a consistent gain in every seed. DQN was sensitive to the learning rate and target copy interval in CartPole. In Acrobot, the high learning rate failed in 2/3 final evaluations across seeds. Both extreme exploration durations succeeded in 2/3 seeds, versus 3/3 for the base.

**What did the ablation reveal?** With other components fixed, removing the target network reduced final performance to $-500$ for every seed. This supports its use in the tested configuration. Replay stayed active, so its contribution cannot be measured from these runs.

### Limitations

Only three seeds were used per main configuration. Standard deviations and bands provide descriptive measures. The study did not test statistical significance or independently evaluate configurations chosen after diagnostics. The CartPole tuning used seeds already involved in the search, so they do not constitute an external test set.

The training logs include returns but omit losses, Q-value magnitudes, gradients, tabular occupancy, and state coverage. Code review found that the core operations are consistent with Q-learning and DQN. Establishing correctness formally and identifying the exact mechanism of deterioration would require additional evidence.

The study does not test interactions between hyperparameters, larger budgets, alternate ranges, or wider networks. Tables generalize only within their partition, and DQN does not guarantee safe extrapolation. Timing was measured without controlling temperature and system load. One visual episode, favorable or unfavorable, cannot represent aggregate performance.

## 10. Conclusions

Tabular Q-learning was computationally inexpensive in CartPole, but all discretizations fell below the final threshold. Finer resolution did not ensure improvement per seed. The original DQN deteriorated. Subsequent tuning improved mean return, although only 1/3 seeds solved the environment. The improvement remained partial across seeds.

In Acrobot, the base configuration achieved $-83.03 \pm 5.55$ with 3/3 successes. The low rate kept 3/3 successes with lower final dispersion. The high rate deteriorated in two seeds. Neither alternate exploration duration exceeded the base. Without a target network, the three final evaluations ended at $-500$. These conclusions apply to the tested budgets and implementation.

## 11. Reproducibility and traceability

Each seed folder contains `config.json`, `training.csv`, `evaluation.csv`, `summary.json`, and `model.pt` or `q.npy`. The summary includes one hundred returns, execution times, and provisional crossings. `requirements_lock.txt` records installed versions. `plot.py` generates the original figures. `plot_revision.py` adds the diagnostics and tuned results alongside the original data.

Run `run.py -suite tier1` and `run.py -suite tier2` for the main experiments. Tuning uses `configs()["cart_dqn"]` with `lr=0.0001` and `target_every=10000`. Execute it through `run(...)` with seeds 11, 22, and 33 and output to `results_diagnostico`. Generate the GIFs with `video.py` and each model path.

Existing runs with a `summary.json` file are skipped. To reproduce from scratch, use a new `-out` folder or a clean project copy. Final weights support evaluation. A full resume checkpoint would also require replay and optimizer state. Repository publication and its link are separate steps.

## 12. AI use statement

IA was used to analyze requirements, propose commands, review code and logs, guide diagnostics, and assist with writing. The student ran the training on their computer. The re port's numerical results were checked against the sup plied CSVs, configurations, and one hundred final re turns per run.

### Appendix A. Main prompts

The following requests from the workflow are translated into English. They served as work instructions.

1. "Give me all the tools or files needed to run this and what I should expect."
2. "Let's go step by step so we don't make mistakes, but also explain what I need to do in detail."
3. "For each thing we do, also tell me roughly what percentage of the whole project we have completed."

Installation, training, diagnostic, and summary outputs were also provided, together with ZIP files for review. The numerical results in this report come from the actual runs. The earlier report of estimates served a separate purpose. The oral defense should cover the code, figures, and modifications, including those made with assistance.

## References


[1] Farama Foundation. *Cart Pole, Gymnasium 1.2.3.*

[2] Farama Foundation. *Acrobot, Gymnasium 1.2.3.*

[3] Farama Foundation (2023). *Deep Dive: Gymnasium Step API.*

[4] Farama Foundation. Gymnasium 1.2.3 action and observation documentation for Bipedal Walker, Hopper, Half Cheetah, and Ant.

[5] OpenAI. *Spinning Up: Proximal Policy Optimization.*

[6] OpenAI. *Spinning Up: Soft Actor-Critic.*

[7] OpenAI. *Spinning Up: Twin Delayed DDPG.*

[8] Mnih, V. et al. (2013). *Playing Atari with Deep Reinforcement Learning.* arXiv:1312.5602.